

Deep Sea Mining – Relatório

ANO LETIVO 2025/2026
DISCIPLINA PROGRAMAÇÃO AVANÇADA

1 DE JUNHO, 2026

Inês Moreira – 2019130354
Ivo Gomes – 2019143183
João Barão - 2024136885

Instituto Superior de Engenharia
Politécnico de Coimbra

Índice

Índice *2*

Introdução *3*

Funcionamento do Jogo *4*

Arquitetura do Jogo *6*

Modelo de Dados 6

Diagrama de Classes 8

Máquina de estados 9

Diagrama de Estados 10

Facade (*DeepSeaManager*) 10

Interface Gráfica *12*

Menu Inicial 12

MenuBar 12

Superficie 13

Fosso 13

Fundo 14

Puzzle 14

Tarefas Realizadas - Tabela *14*

Conclusão *15*

Introdução

Este relatório documenta o desenvolvimento de um jogo de exploração intitulado "**Deep Sea Mining**", realizado no âmbito da disciplina de **Programação Avançada**. O jogo coloca o utilizador no comando de um Navio à superfície do oceano, a partir do qual controla Drones subaquáticos numa missão para recuperar artefactos de uma civilização antiga. A gestão rigorosa do combustível e a manutenção da integridade dos equipamentos são essenciais para o sucesso da missão. Este documento detalha a arquitetura de software implementada, que garante uma separação clara entre o Modelo de Dados e a Interface Gráfica. O projeto foi desenvolvido em **Java**, utilizando **JavaFX** para a UI e padrões de design como a **Máquina de Estados** e o padrão **Facade**.

Funcionamento do Jogo

O jogo Deep Sea Mining consiste em exploração oceânica profunda. O jogador assume o comando de um **Navio** cientifico que se encontra posicionado na superfície do oceano a partir do qual controla a uma frota de **Drones** subaquáticos cujo objetivo é detetar factos perdidos de uma civilização antiga tendo em conta a integridade dos equipamentos e o combustível que se encontra disponível.

Experiência do utilizador está estruturada em três ambientes diferentes a **Superfície,** o **Fosso** e o **Fundo**:



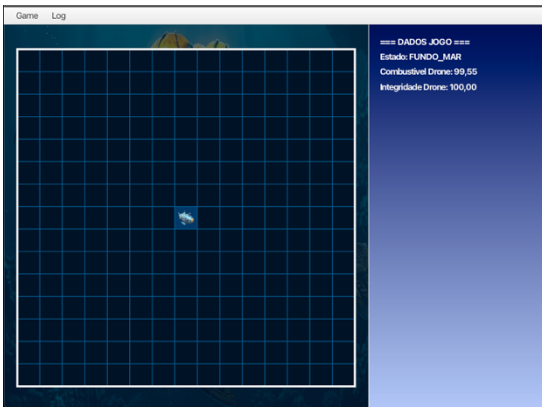
- A **Superfície**:
  - Representa a área que o navio navega, e é modelada por uma **grelha bidimensional**.
  - Serve como base de operações, abrigando a **Oficina**. Aqui, o navio pode reparar danos sofridos pelos **Drones** durante a expedição, usando recursos recolhidos. Também pode melhorar o casco e o tanque, bem como reabastecer.
  - É aqui que começa a cada mergulho bem como, o local onde as recolhas finais são validadas para a pontuação.



- **A Fosso** (Fase de **Subida/Descida**)
  - É uma grelha bidimensional vertical que liga a **Superfície** ao **Fundo**.
  - O movimento selecionado é predominantemente vertical, permitindo ao **Drone** descer ao fundo do mar e subir de volta à superfície para o navio, potencialmente com carga.
  - O **Fosso** possui correntes marítimas que empurraram o **Drone** e duplicam o seu consumo de combustível. Além disso, apresenta perigos físicos como **rochas** e **animais marinhos**.

- De forma simular uma pressão subaquática maior e tornar o jogo mais desafiante o dano provocado pelos impactos ao longo da descida no **Fosso** utiliza a sequência matemática de números primos começa em 1 a e cada invocação avança para número primo seguinte aumentando assim a letalidade do ambiente ao longo do tempo.

- O **Fundo**:
  - A área do jogo é inicialmente apresentada coberta por uma névoa, e é à medida que o **Drone** se desloca no **Fundo** que vai-se revelando os blocos do mapa. Nesta camada que inicialmente está escondida vai-se encontrar **minerais** e os outros **artefactos** arqueológicos.



  - O terreno contém também monstros marinhos. O Drone necessita de desviar dos mesmos.
  - Quando alcança o artefacto é lhe proposto um minijogo que tem de ganhar para levar consigo o artefacto.

Arquitetura do Jogo

A arquitetura do Deep Sea Mining separa o lógica do **Modelo de Dados** da **Interface Gráfica**, seguindo desta forma os princípios de engenharia de software. A lógica é gerida pela **Máquina de Estados**, onde é feita a transição entre as diferentes fases do jogo.

A comunicação entre a **Interface Gráfica** e o **Modelo de Dados** é gerida pelo **DeepSeaManager**. Esta classe atua como um gestor que expõe os métodos que são necessários para a **Interface Gráfica**, delegando os pedidos operacionais diretamente ao contexto da **Máquina de Estados**, o **DeepSeaContext**. Assim a **UI**, nunca acede ou manipula o **Modelo de Dados** sem passar pelas verificações da **Máquina de Estados**.

O **DeepSeaContext** gere a Máquina de Estados, conectando o estado ativo, **IDeepSeaState**, aos dados do jogo. O método **changeState()** permite transições de estados à medida que o jogo vai avançando, ao mesmo tempo vai registando no histórico do **DeepSeaLog.**

Modelo de Dados

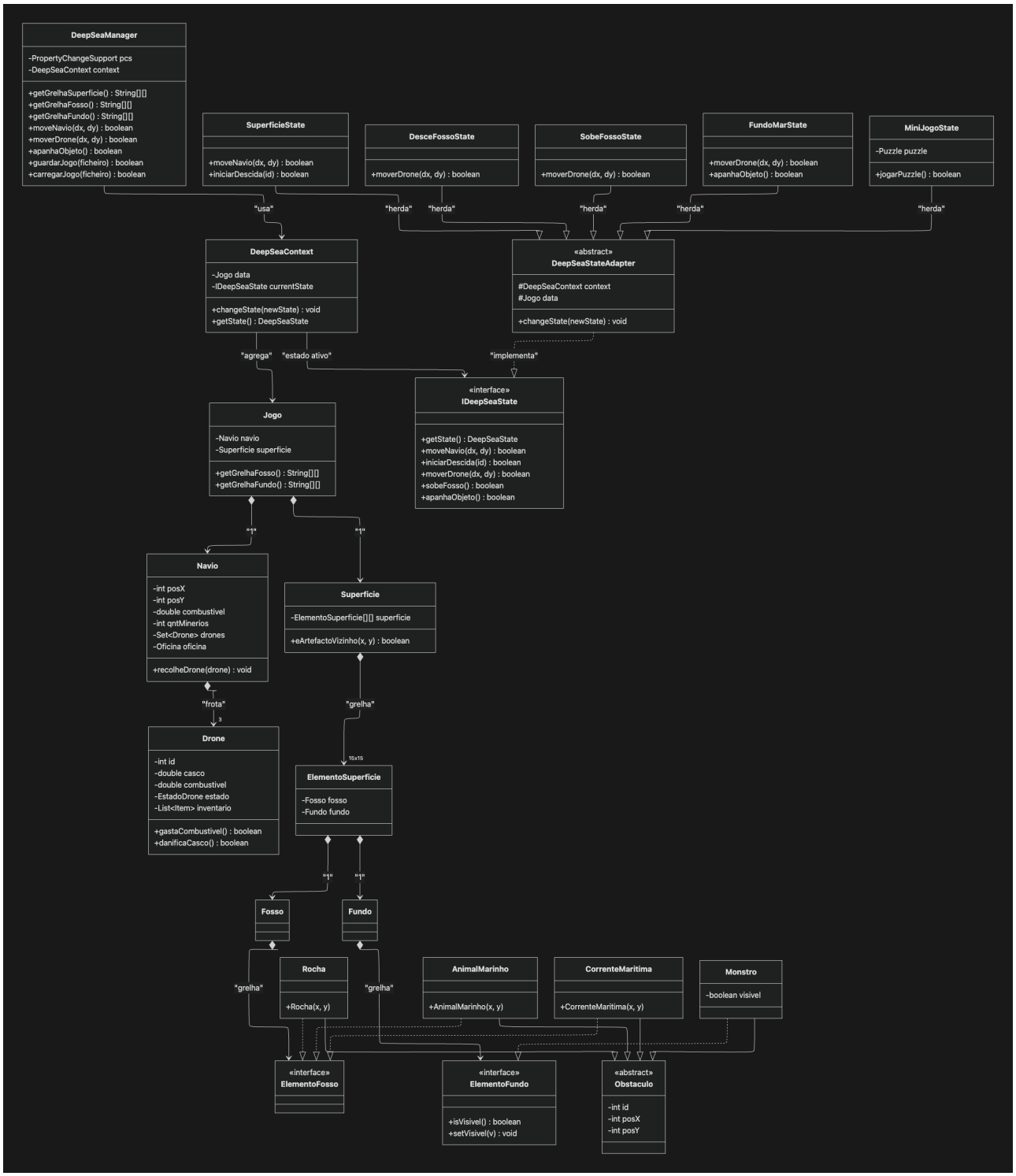
O **Modelo de Dados** está organizado em volta de em classe central que controla o estados e as entidades do jogo, conectando os elementos de exploração aos mapas e recursos.

As classes principais são:

- O **Jogo** é a classe central que contem todos os dados da simulação. Gere as instâncias do **Navio**, do **Drone**, e as três áreas onde a ação ocorre: a **Superficie**, o **Fosso** e o **Fundo**.
- O **Navio** é a base de operações na Superfice. Gere a **Oficina**, onde estão guardados os recursos que foram recolhidos ao longo do jogo e onde o jogador pode gastar os minérios que encontrou para melhorar e/ou reparar os Drones.
- O **Drone** é o veículo controlado pelo utilizador durante a descida e subida do **Fosso** e no **Fundo** do Mar. Monitoriza os níveis de combustível, a integridade do casco, a capacidade máxima de carga e os tipos de itens transportados no momento.
- A **Superficie** é uma matriz bidimensional horizontal que serve como campo de jogo para o navio e valida as recolhas feitas.

- O **Fosso** é uma matriz bidimensional vertical que sofre desgaste progressivo e apresenta perigos como rochas, animais marinhos e correntes marítimas.
- O **Fundo** é um mapa horizontal oculto por uma “névoa”, que contém **Minérios**, perigos como o **Monstro** e **Artefactos**.
- Os elementos que é possível colecionar e os obstáculos:
    - Os **Colecionáveis:**
        - Os **Minérios** que servem como moeda para gastar em melhorias.
        - **Os Artefactos** que desbloqueiam o **Puzzle** necessário para a sua recolha bem-sucedida.
    - Os **Obstáculos**: As classes **ElementoFosso** e **ElementoFundo** definem as penalizações de combustível, o dano direto sofrido pelo casco do Drone e as ameaças do ambiente.

Diagrama de Classes



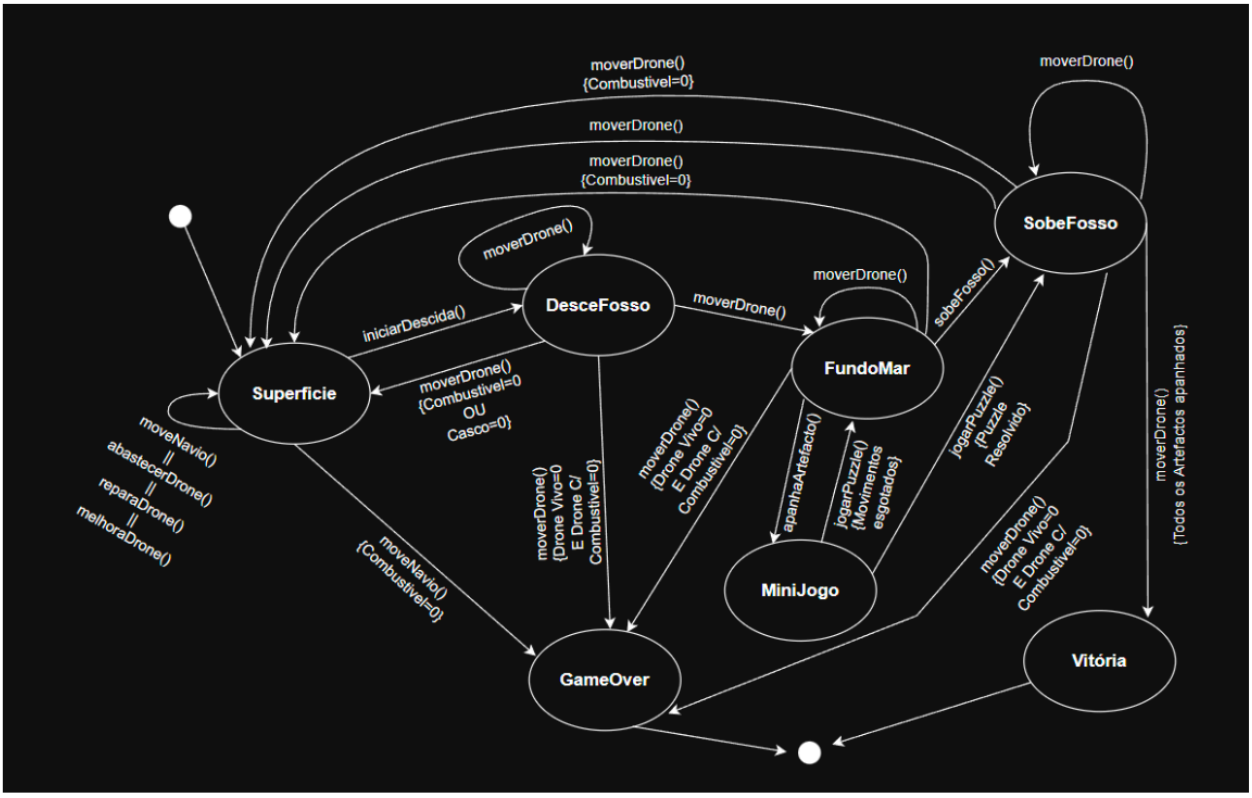
Máquina de estados

A **Máquina de Estados** consiste numa por uma hierarquia, onde os comportamentos são específicos aos estados:

- **SuperficieState –** Estado em que o jogo se inicia, onde o jogador consegue navegar com o **Navio** e onde consegue interagir com a **Oficina** para eventualmente fazer melhorias, reparações e reabastecimento dos **Drones.**
- **DesceFossoState** e **SobeFossoState –** Geram a fase a intermédia, implementado as regras de pressão, o dano por impacto vai aumentando através da sequência matemática de números primos à medida que o **Drone** vai descendo, e a resistência acrescida dada pelas correntes que duplicam o consumo de combustível.
- **FundoMarState –** Lida com a exploração em ambiente desconhecido e que está originalmente escondido sob "névoa", gerindo de forma gradual, a revelação do mapa à medida que o **Drone** se desloca e o desvio de perigos ou monstros marinhos.
- **MiniJogoState –** Assuma o controlo do **Drone** quando este atingir um artefacto, iniciando assim um mini jogo que permite ao jogador recolhê-lo e levá-lo consigo.

- **VitoriaState** e **GameOverState –** Os estados finais do jogo são ativados quando o jogador recolhe todos os artefactos, resultando num **VitoriaState**, ou quando a integridade dos Drones e do Navio é comprometida, levando a um **GameOverState**.

Diagrama de Estados



Facade (*DeepSeaManager*)

Esta classe, **DeepSeaManager**, implementa o padrão de desenho **Facade**, e funciona como ponto chave para simplificar a interação entre a **UI** e a logica interna do jogo. Na vez de a Interface Gráfica interagir de forma direta com o **Modelo de dados** ou com a **Maquina de Estados**, a **UI** invoca os métodos expostos pela classe **DeepSeaManager**.

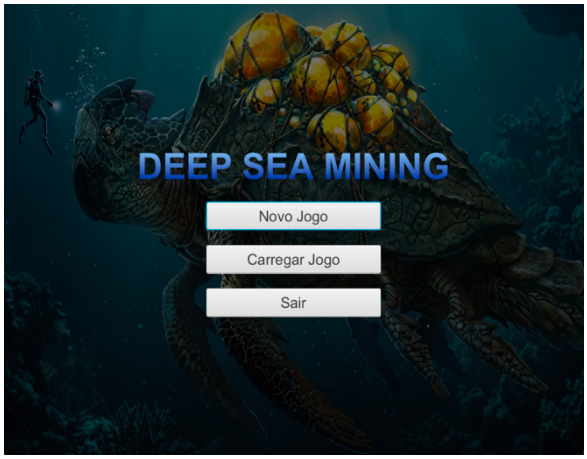
As responsabilidades fundamentais desta **Facade** são:

- Encaminhar os comandos do jogador, como moverNavio(), moverDrone() e iniciarDescida(), diretamente para o DeepSeaContext. Esta abordagem centraliza as verificações e as alterações do estado do jogo.
- Descrever métodos simples para a UI aceder à informação, como obter as grelhas visuais da Superficie, do Fosso e do Fundo, já formatadas como matrizes, consultar o estado do Drone ou o seu inventário.

- A classe é responsável pela gravação e carregamento do jogo através dos métodos guardarJogo() e carregarJogo(). Utiliza serialização direta do objeto DeepSeaContext e recalibra os *id*s acumulados nas classes Drone e Obstáculo durante a leitura.
- Integra a classe de utilidade ProprietyChangeSupport para registar escutas da UI e disparar o evento _update_ sempre que uma ação lógica altera o estado do jogo. Isto permite que a UI atualize independentemente do fluxo principal.

Interface Gráfica

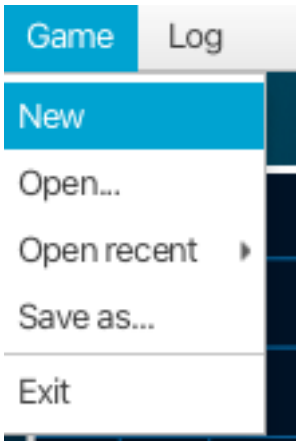
Menu Inicial



Título: Identificação do jogo como "Deep Sea Mining".
Botão "Novo Jogo": Ação que inicializa a simulação e define a Máquina de Estados no estado de Superfície.
Botão "Carregar Jogo": Abre uma janela de seleção de ficheiro, **FileChooser,** para restaurar um jogo gravado em ficheiro binário.
Botão "Sair": Fecha a aplicação.

MenuBar

Menu "Ficheiro":



- **New**: Permite reiniciar a simulação a qualquer momento.
- **Open**: Importa um ficheiro binário existente para restaurar apartir desse mesmo ficheiro.
- **Open Recent**: Lista e permite continuar rapidamente um dos **últimos 5 jogos guardados** no computador.
- **Save as**: Exporta o estado atual e grava em ficheiro binário.
- **Exit**: Encerra o programa.

Superficie



Área Central (Canvas):

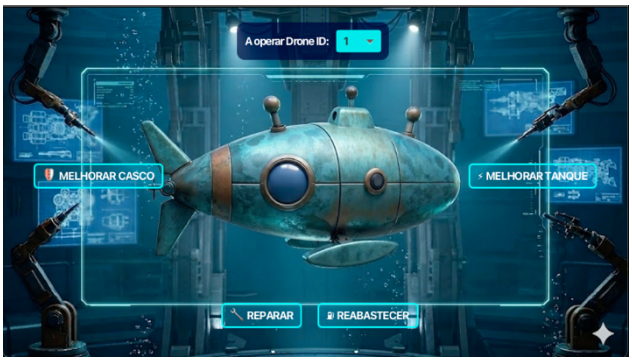
- Renderização gráfica da grelha de superfície.
- Representação do **Navio** posicionado na célula central.

Painel Lateral da Superficie:

- Indicador do combustível geral do **Navio**.
- Quantidade de minérios depositados com sucesso no Navio.

Janela da Oficina :

- *Lista de Drones*: Seletor visual da frota disponível.



- *Botão "Reabastecer"*: Utiliza o combustível do Navio para atestar o tanque do Drone selecionado.
- *Botões "Melhor Carcos"*e *"Melhorar Tanque"*: Opções para gastar minérios na melhoria do casco ou da capacidade de carga máxima.
- O botão "Reparar": Serve para restaurar a integridade do casco do Drone selecionado que tenha sofrido desgaste, consumindo uma quantidade proporcional do combustível armazenado no navio.

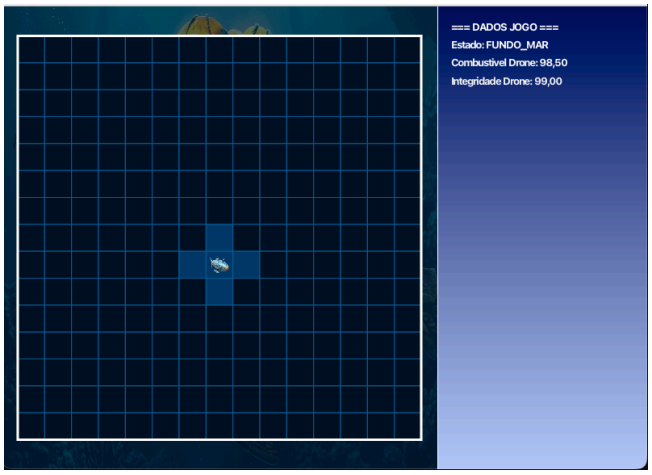
Fosso



Grelha Bidimensional Vertical (Canvas): Painel onde o jogador controla as coordenadas do Drone na descida ou subida.
Rochas: Visualização gráfica das paredes laterais geradas aleatoriamente a cada degrau de profundidade.
Perigos do Fosso: Representações visuais de animais marinhos e de correntes marítimas.
Painel Lateral do Fosso: Indicador do estado físico e do combustivel do Drone.

Fundo

Grelha do Fundo (Canvas): Mapa horizontal oculto inicialmente por “névoa” onde o drone navega, revelando apenas as células por onde o drone passa.
Obstáculos: Representação de monstros marinhos que causam danos por impacto ao casco.
Recursos: Localizações contendo depósitos de minério e artefactos.
Painel Lateral do Fundo: Indicador do estado físico e do combustivel do Drone.



Puzzle

Grelha do Puzzle (4x4):Mostra 15 peças numeradas dispostas numa matriz 4x4, com um espaço livre para deslizar uma peça.
Contador de Movimentos Restantes: Mostra o decréscimo de tentativas restantes.



Tarefas Realizadas

Etapa	Funcionalidade	Estado
Etapa 1	Modelo de Grelhas (Matrizes)	Concluído
	Entidades Navio e Drone	Concluído
	Testes Unitários (Etapa 1)	Concluído
Etapa 2	Máquina de Estados Finita (FSM)	Concluído
	DeepSeaManager	Concluído
	Persistência de Dados	Concluído
	Gestão de Registos (DeepSeaLog)	Concluído
	Testes Unitários (Etapa 2)	Concluido
Etapa 3	Interface Gráfica JavaFX	Concluído
	Espaço do Jogo em Canvas	Concluído
	Painéis Laterais de Status	Concluído
Etapa Final	SoundManager	Concluído
	Suporte Multi-Janela	Concluído

Conclusão

O desenvolvimento do “Deep Sea Mining” permitiu aplicar com sucesso os conceitos e padrões de desenho abordados na disciplina. A criação da Máquina de Estados e do padrão Facade foi crucial para centralizar a comunicação e garantir que a Interface Gráfica nunca manipula o Modelo de Dados diretamente. Todas as etapas de desenvolvimento planeadas foram concluídas, cumprindo assim os requisitos propostos para este projeto.